Forecasting Coffee Shop Ingredient Usage with AR(2)

1st Moritz Kuberek
Department of Informatics
Universität Hamburg
Hamburg, Germany
moritz.kuberek@studium.uni-hamburg.de

2nd Leon Richter
Department of Informatics
Universität Hamburg
Hamburg, Germany
leon.richter@studium.uni-hamburg.de

***Abstract*—We demonstrate a database-backed forecasting workflow for estimating future ingredient usage in a coffee shop scenario. The system combines relational sales data from PostgreSQL with recipe documents stored in MongoDB, derives daily ingredient usage time series, and applies a self-implemented AR(2) autoregressive forecasting model. The demo shows how users can switch between ingredients such as milk and sugar, adjust model parameters, generate future forecasts, and evaluate the model on held-out historical data. This is relevant because ingredient-level forecasts can support purchasing decisions, reduce shortages, and make operational planning more transparent.**

***Index Terms*—forecasting, time series, autoregression, AR(2), PostgreSQL, MongoDB, ingredient usage, coffee sales**

I. Introduction

Coffee shops need to plan ingredient purchases before the actual demand is known. If too little milk, sugar, or coffee is available, products may become unavailable during opening hours. If too much is purchased, storage costs and waste may increase. Forecasting ingredient usage is therefore a relevant operational problem, especially when sales data and recipe data are stored in different systems.

We address this problem with a small demo system that combines relational and document-oriented data sources. Sales transactions are stored in PostgreSQL, while coffee recipes and ingredient amounts are stored in MongoDB. The system aggregates both sources into a daily ingredient usage time series and applies a self-implemented AR(2) forecasting model. The demo focuses on transparency rather than maximum prediction accuracy: users can see how the data flows through the system, change model parameters, select different ingredients, and compare forecasts with historical observations.

II. System/Solution/Architecture Overview

Figure 1 shows the architecture of the demo system. The relational database stores coffee sales in the tables `coffee_sales` and `coffees`. These tables provide the sale date and the sold coffee specialty. The NoSQL database stores the completed recipe dataset in a MongoDB `recipes` collection. Each recipe document contains the coffee name, ingredients, ingredient amounts, units, equipment, and preparation steps.

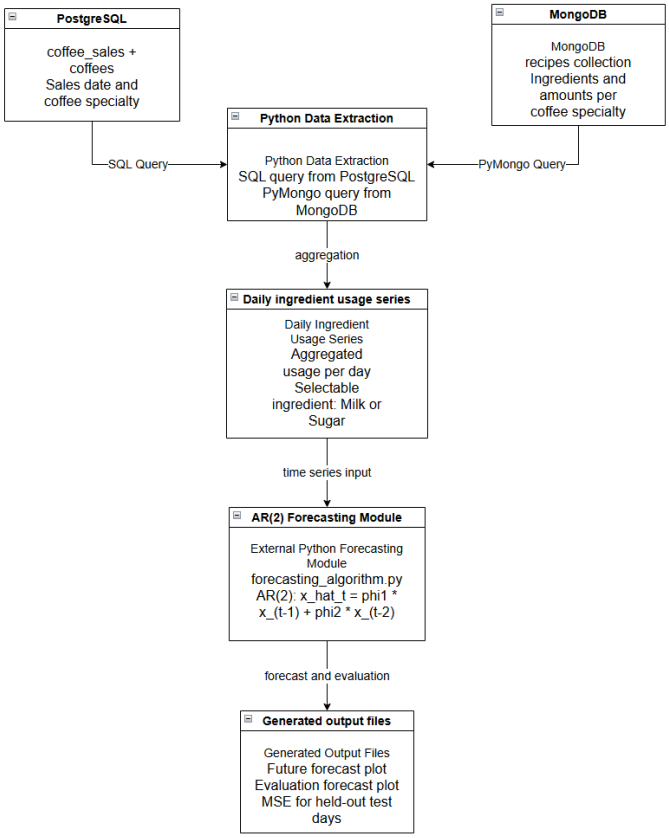

Fig. 1. Architecture of the AR(2)-based ingredient forecasting demo. PostgreSQL provides sales data, MongoDB provides recipe data, and the external Python script performs aggregation, forecasting, evaluation, and plot generation.

The Python script first reads all sales from PostgreSQL using a SQL join between sales and coffee names. It then reads the recipe documents from MongoDB using PyMongo. For each sale, the script identifies the corresponding recipe and adds the selected ingredient amount to the respective sale date. This creates one ordered time series per ingredient, for example daily milk usage or daily sugar usage.

The forecasting component is implemented in a separate file, `forecasting_algorithm.py`. This file contains only the forecasting logic and no database access, plotting, or export

code. We use an AR(2) autoregressive model, which predicts the next value from the two previous values:

$$\hat{x}_t = \phi_1 x_{t-1} + \phi_2 x_{t-2}.$$

Here, ϕ_1 and ϕ_2 are user-adjustable parameters. This model is simple, interpretable, and suitable for demonstration because each prediction can be explained directly from the previous two days. Autoregressive models are a standard class of time series forecasting models [1].

III. Demonstration Proposal/Scenarios/Workflow

The demo presents the complete workflow from database extraction to forecast generation. A visitor can select an ingredient, adjust the AR(2) parameters, run the script, and inspect the resulting forecast and evaluation plots. The demonstration is designed to make the relation between database contents, aggregation logic, model parameters, and final forecast visible.

A. Setup

The demo was implemented on Windows 11 using Python 3.13 as the programming language. PostgreSQL 18 was used as the relational database system and MongoDB 8.3.2 was used as the NoSQL database system. The demo was executed on a local laptop with [11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz (2.30 GHz)] and 16 GB of main memory. The Python environment uses `psycopg2` for PostgreSQL access, `pymongo` for MongoDB access, and `matplotlib` for plot generation.

The relational part contains the sales data from the previous exercise sheets. The document-oriented part contains the completed recipe dataset, including the previously missing recipe entries for Cappuccino and Espresso. The forecasting script is executed externally from the command line. A typical execution for milk usage is:

```
python exercise3_forecast_ar2.py
--ingredient Milk
  --phi1 0.7 --phi2 0.3 --horizon 7
  --test-size 14
```

The same workflow can be executed for sugar by replacing `Milk` with `Sugar`. The parameters `phi1` and `phi2` directly control the autoregressive weighting of the previous two days.

B. User Experience

During the demo, the visitor first sees the architecture and the two involved database systems. We then explain how a sale is transformed into ingredient usage: for example, if a sold coffee recipe contains 200 ml of milk, the daily milk total is increased by 200 ml. This step connects the relational sales table with the MongoDB recipe documents.

Next, the visitor can choose an ingredient and parameter setting. The script produces two plots automatically. The demo generates both a future forecast plot and an evaluation plot. In the paper, we show the evaluation plot because it directly illustrates how the AR(2) predictions compare to held-out historical observations.

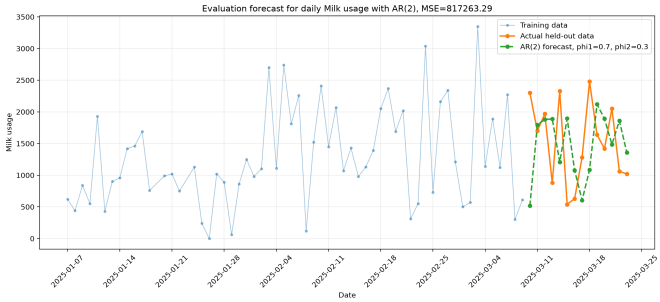

Fig. 2. Evaluation of the AR(2) model for daily milk usage. The last historical days are held out, predicted step by step, and compared with the actual observed values.

Visitors can interact with the demo by changing the selected ingredient, changing ϕ_1 and ϕ_2, and observing how the future forecast and evaluation plot change. This makes the demo suitable for discussing both database integration and the behavior of a simple time series model.

C. Conclusion

The demo shows how relational sales data and document-oriented recipe data can be combined to forecast ingredient usage. Its main strength is transparency: every processing step from database extraction to daily aggregation, AR(2) forecasting, and evaluation is visible and reproducible. The system is not intended as a production-grade forecasting tool, but as a compact demonstration of database integration and interpretable time series forecasting.

References

[1] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.